

Figure 10-10. *EventSchedule composition*

Using Classes

Relationships also exist between classes where there is no related structure: no common parent or shared interface. It is quite common for objects to be passed around a system without needing to store references to them everywhere (in fact, it's better not to). These are modeled as *dependencies* in the UML (and frequently referred to as such when discussing OO software), and they occur when class methods require other classes as parameters or return other classes.

Dependency relationships are indicated with a dotted line and open arrowhead. Perhaps our bookshop data model now includes a module that makes queries on `ShopInventory` instances to prepare stock reports. `StockTaker` does not need to store a reference to `ShopInventory` to do this; it just needs an `Inventory` interface implementation to work with.

Figure 10-11 shows the relationship between the three classes, `StockTaker`, `ShopInventory`, and `WarehouseInventory`, and the `Inventory` interface. The inventory classes implement the `Inventory` interface (dotted line with unfilled solid arrow), and `StockTaker` has a dependency on `Inventory` (dotted line, open arrow).

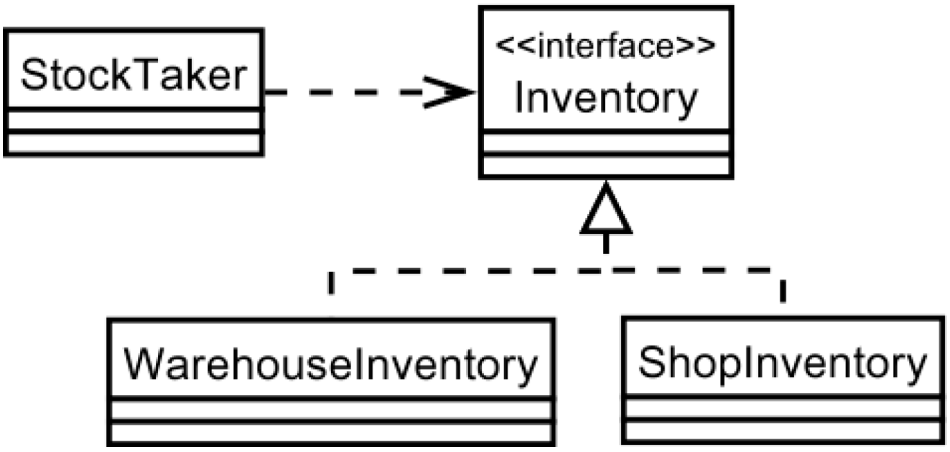


Figure 10-11. *StockTaker dependency on Inventory*